UNIVERSIDADE DO VALE DO RIO DOS SINOS — UNISINOS
UNIDADE ACADÊMICA DE GRADUAÇÃO OU UNIDADE ACADÊMICA DE EDUCAÇÃO ONLINE
BACHARELADO EM SISTEMAS DE INFORMAÇÃO
GRADUAÇÃO

JULIANA ROCHA FERNANDES

ESTUDO COMPARATIVO ENTRE DUAS DIFERENTES ESTRATÉGIAS DE CONSTRUÇÃO DE PÁGINAS WEB: RENDERIZAÇÃO NO SERVER SIDE, E RENDERIZAÇÃO NO CLIENT SIDE

Impactos no desempenho

SÃO LEOPOLDO
2025

Juliana Rocha Fernandes

ESTUDO COMPARATIVO ENTRE DUAS DIFERENTES ESTRATÉGIAS DE CONSTRUÇÃO DE PÁGINAS WEB: RENDERIZAÇÃO NO SERVER SIDE, E RENDERIZAÇÃO NO CLIENT SIDE

Impactos no desempenho

Trabalho de Conclusão de Curso apresentado como requisito parcial para a obtenção do título de Bacharel em Sistemas de Informação, pelo Curso de Sistemas de Informação da Universidade do Vale do Rio dos Sinos — UNISINOS

Orientador:
Prof. Guilherme Lacerda

São Leopoldo
2025

Aos nossos pais.

If I have seen farther than others,
it is because I stood on the shoulders of giants.
— SIR ISAAC NEWTON

That brain of mine is something more than merely mortal,
as time will show.
— ADA LOVELACE

AGRADECIMENTOS

Sou profundamente grata aos meus pais, que sempre me ofereceram oportunidades e apoio em cada etapa até este momento da minha trajetória acadêmica. À minha mãe, Marileia Alves Rocha, pelo apoio constante às minhas decisões. Ao meu pai, Gerson Severo Fernandes, por me permitir sonhar e me proporcionar uma pitada de conforto para a realização da minha graduação. Ao meu irmão, Alexandre Davi Rocha Fernandes, pela generosidade em compartilhar seu conhecimento nos momentos mais críticos. Aos professores que me mostraram que há espaço para mim na vida acadêmica. Estendo também minha gratidão às pessoas que tive o privilégio de conhecer ao longo da minha jornada profissional, bem como à comunidade acadêmica, em especial à FACIN/PUCRS e à Unisinos.

"*Ninguém abre um livro sem que aprenda alguma coisa*".
(Anônimo)

RESUMO

Este trabalho apresenta uma proposta de comparação entre abordagens de renderização *server-side* e *client-side* no contexto de aplicações web modernas, utilizando o framework **Ruby on Rails** com **Hotwire** e uma implementação equivalente com **React**. O estudo busca avaliar diferenças de desempenho, consumo de recursos e experiência do usuário, empregando métricas reconhecidas como *Largest Contentful Paint* (LCP), *First Contentful Paint* (FCP), *Interaction to Next Paint* (INP) e *Cumulative Layout Shift* (CLS). Os experimentos são conduzidos em ambiente controlado, com medições realizadas por meio do *Google PageSpeed Insights* e do *Chrome DevTools*. Espera-se que os resultados indiquem vantagens distintas em cada abordagem: o *server-side rendering* tende a oferecer melhor desempenho inicial e otimização para mecanismos de busca, enquanto o *client-side rendering* se destaca pela interatividade e flexibilidade na construção de interfaces dinâmicas. O estudo visa contribuir para decisões arquiteturais mais embasadas no desenvolvimento de aplicações web.

Palavras-chave: Renderização no servidor. Renderização no cliente. Hotwire. React. Desempenho web. Experiência do desenvolvimento. SEO. Acessibilidade. Desenvolvimento web moderno.

ABSTRACT

This study presents a comparative analysis between server-side and client-side rendering approaches in modern web applications, using the **Ruby on Rails** framework with **Hotwire** and an equivalent implementation with **React**. The research evaluates performance, resource consumption, and user experience, employing well-known metrics such as *Largest Contentful Paint* (LCP), *First Contentful Paint* (FCP), *Interaction to Next Paint* (INP), and *Cumulative Layout Shift* (CLS). Measurements were performed using *Google PageSpeed Insights* and *Chrome DevTools*. Results are expected to show that server-side rendering provides better initial performance and search engine optimization, while client-side rendering offers superior interactivity and flexibility for building dynamic interfaces. The study aims to support better-informed architectural decisions in web application development.

Keywords: Server-side rendering. Client-side rendering. Hotwire. React. Web performance. User experience. SEO. Accessibility. Modern web development.

LISTA DE FIGURAS

1 Linha do tempo das arquiteturas de renderização 15
2 Fluxo tradicional de SSR (Adaptado de (LINDEN, 2023)) 16
3 Fluxo típico de CSR: renderização no cliente com dados da API (adaptado de (S. BEZERRA, 2024)) 17

LISTA DE TABELAS

1 Vantagens da Arquitetura CSR 18

2 Comparativo de desempenho entre SSR (Hotwire) e CSR (React) — valores estimados 25

LISTA DE SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
API	Interface de Programação de Aplicações
CSR	Renderização do Lado do Cliente
DOM	Modelo de Objeto de Documento
FCP	Primeira Exibição de Conteúdo
HTML	Linguagem de Marcação de Hipertexto
INP	Interação até a Próxima Renderização
JSON	Notação de Objetos JavaScript
LCP	Maior Exibição de Conteúdo
MVC	Modelo-Visão-Controlador
SEO	Otimização para Motores de Busca
SPA	Aplicação de Página Única
SSR	Renderização do Lado do Servidor
HTML	Linguagem de Marcação de Hipertexto
UNISINOS	Universidade do Vale do Rio dos Sinos

SUMÁRIO

1 INTRODUÇÃO **11**
1.1 Questão norteadora 12
1.2 Objetivos 12
1.2.1 Objetivo geral 12
1.2.2 Objetivos específicos 12
1.3 Justificativa 13

2 REFERENCIAL TEÓRICO **14**
2.1 Arquiteturas de Renderização *Web* 14

3 METODOLOGIA **21**
3.1 Procedimentos de Desenvolvimento 21
3.2 Estratégias de Avaliação de Desempenho 21
3.3 Justificativa das Métricas de Avaliação 22

4 RESULTADOS ESPERADOS **24**
4.0.1 Analise 24
4.0.2 Resultados 25

5 CONCLUSÃO **27**

REFERÊNCIAS **28**

1 INTRODUÇÃO

No universo do desenvolvimento web contemporâneo, a escolha da estratégia de renderização é fundamental. Esta decisão afeta diretamente o desempenho das aplicações. Impacta também a experiência do usuário final. O presente estudo visa comparar duas abordagens predominantes: a renderização no servidor (*server-side*) e a renderização no cliente (*client-side*), conforme discutido por (NORDSTRöM; DIXELIUS, 2023). Cada uma dessas estratégias apresenta características distintas e vantagens específicas para diferentes cenários de implementação (KORA; MANITA, 2024).

A renderização *server-side*, representada neste estudo pelo *framework* Hotwire, oferece uma abordagem interessante. O Hotwire integra tecnologias como Stimulus e Turbo. Estas ferramentas trabalham em conjunto para processar o conteúdo no servidor. Apenas o HTML resultante é enviado ao navegador do usuário (OLLILA; MäKITALO; MIKKONEN, 2022). Esta abordagem possui raízes históricas na web tradicional. Porém, ganhou novas funcionalidades com *frameworks* modernos que buscam equilibrar o processamento no servidor com uma experiência dinâmica (SRIVASTAVA et al., 2024).

Por outro lado, a renderização *client-side*, exemplificada pelo React, transfere a responsabilidade para o navegador. O React, combinado com APIs, permite a construção de interfaces altamente interativas (KISHORE; MAHENDRA, 2020). O servidor fornece principalmente dados brutos via APIs. O processamento e a renderização acontecem inteiramente no dispositivo do usuário. Esta abordagem ganhou popularidade com o advento das *Single Page Applications* (SPAs). Ela promete uma experiência mais fluida e semelhante a aplicações nativas (NORDSTRöM; DIXELIUS, 2023).

Ambas as estratégias apresentam implicações significativas para o desenvolvimento web. Fatores como tempo de carregamento inicial, performance percebida e SEO são diretamente afetados (SCHNEIDER, 2016). A manutenção e escalabilidade também variam consideravelmente entre as abordagens. A escolha entre *server-side* e *client-side* frequentemente depende do contexto específico da aplicação. Requer uma análise cuidadosa das necessidades do projeto e das limitações técnicas existentes (DAMKE; GREGORINI, 2024a).

O desenvolvimento de software web contemporâneo enfrenta diversos desafios técnicos e operacionais. Tempos de carregamento excessivamente altos frustram os usuários e aumentam as taxas de rejeição (JARTARGHAR et al., 2022). O desperdício de recursos computacionais eleva custos operacionais desnecessariamente. Ciclos de retrabalho constantes diminuem a produtividade das equipes de desenvolvimento. Estes problemas não são isolados, mas interconectados, afetando diretamente tanto a experiência do usuário quanto a eficiência operacional das organizações (ZILIOTTI, 2024).

A arquitetura escolhida para uma aplicação web influencia diretamente estes fatores problemáticos. A decisão entre renderização no cliente (*Client-Side Rendering* - CSR) ou no servidor (*Server-Side Rendering* - SSR) não é trivial (PINTO; HOFFMANN; URIARTE, 2023).

Esta escolha arquitetural impacta significativamente o desempenho percebido pelos usuários finais. A escalabilidade da aplicação sob diferentes volumes de tráfego também é afetada. A complexidade de implementação e manutenção varia consideravelmente entre as abordagens, exigindo análise cuidadosa das implicações de cada opção (ZILIOTTI, 2024).

A relevância do estudo, se pauta nos aspectos que as decisões técnicas tomadas visando ganhos imediatos frequentemente resultam em complicações futuras. O que inicialmente parece uma solução rápida pode gerar retrabalho extensivo posteriormente. A manutenibilidade do sistema torna-se progressivamente mais difícil com cada atalho implementado (PANWAR, 2024). A evolução natural da aplicação encontra obstáculos técnicos não previstos inicialmente. Forma-se assim uma dívida técnica substancial, cujo pagamento eventual pode ser significativamente mais custoso que o investimento inicial evitado (ZILIOTTI, 2024).

Diante do exposto, este estudo comparativo busca analisar objetivamente os prós e contras de cada abordagem. Serão examinados aspectos como velocidade de carregamento e renderização. A complexidade de implementação também será considerada. O consumo de recursos no servidor versus no cliente constitui outro ponto crucial de análise. A investigação também abordará questões de acessibilidade e otimização para mecanismos de busca. Pretende-se fornecer uma visão abrangente que auxilie na tomada de decisão técnica.

1.1 Questão norteadora

Quais são as diferenças práticas entre as abordagens de renderização *server-side* (com Hotwire) e *client-side* (com React) no desenvolvimento de aplicações web, considerando aspectos de desempenho, manutenibilidade e experiência do usuário?

1.2 Objetivos

1.2.1 Objetivo geral

Comparar as abordagens de renderização *server-side* e *client-side* no desenvolvimento de aplicações web, analisando seus impactos no desempenho da aplicação, na experiência do usuário e na manutenibilidade do sistema.

1.2.2 Objetivos específicos

- Desenvolver duas versões de uma aplicação web com Hotwire (SSR) e React (CSR).
- Comparar o desempenho das aplicações em diferentes cenários de uso.
- Avaliar o consumo de recursos e a experiência do usuário.
- Analisar a manutenibilidade e a experiência dos desenvolvedores.

- Verificar os impactos de cada abordagem em SEO e acessibilidade.

1.3 Justificativa

No cenário atual do desenvolvimento web, onde a performance e a experiência do usuário são fatores decisivos para o sucesso de uma aplicação, a escolha da estratégia de renderização representa um desafio técnico recorrente. A decisão entre utilizar renderização no lado do servidor ou no lado do cliente pode impactar significativamente diversos aspectos da aplicação, como tempo de carregamento, interatividade, escalabilidade, manutenibilidade e até mesmo visibilidade em mecanismos de busca.

Apesar do avanço dos frameworks modernos, ainda há lacunas quanto à compreensão prática das vantagens e limitações de cada abordagem, especialmente quando comparadas em contextos reais e sob diferentes condições de uso. Ao realizar uma análise comparativa com base em dados empíricos e experiências de desenvolvimento, este trabalho busca oferecer subsídios técnicos que auxiliem desenvolvedores, arquitetos de software e equipes de tecnologia na tomada de decisão mais adequada para cada tipo de projeto.

A relevância do estudo, se pauta nos aspectos que as decisões técnicas tomadas visando ganhos imediatos frequentemente resultam em complicações futuras. O que inicialmente parece uma solução rápida pode gerar retrabalho extensivo posteriormente. A manutenibilidade do sistema torna-se progressivamente mais difícil com cada atalho implementado (PANWAR, 2024). A evolução natural da aplicação encontra obstáculos técnicos não previstos inicialmente. Forma-se assim uma dívida técnica substancial, cujo pagamento eventual pode ser significativamente mais custoso que o investimento inicial evitado (ZILIOTTI, 2024).

Diante do exposto, este estudo comparativo busca analisar objetivamente os prós e contras de cada abordagem. Serão examinados aspectos como velocidade de carregamento e renderização. A complexidade de implementação também será considerada. O consumo de recursos no servidor versus no cliente constitui outro ponto crucial de análise. A investigação também abordará questões de acessibilidade e otimização para mecanismos de busca. Pretende-se fornecer uma visão abrangente que auxilie na tomada de decisão técnica.

Além disso, a proposta contribui para o meio acadêmico ao promover um estudo aplicado que integra teoria e prática, por meio da experimentação com frameworks atuais como React e Hotwire. A relevância do tema também se justifica pela crescente demanda por aplicações web eficientes, acessíveis e sustentáveis, considerando o aumento no volume de acessos por dispositivos móveis e as exigências de usuários cada vez mais exigentes.

2 REFERENCIAL TEÓRICO

2.1 Arquiteturas de Renderização *Web*

Historicamente, a renderização de páginas na web foi concebida para ocorrer majoritariamente no lado do servidor. Durante os primeiros anos da Internet, os sites possuíam caráter essencialmente estático, com conteúdo pouco dinâmico e estruturado em arquivos HTML simples. Sempre que uma página era requisitada, o servidor gerava e enviava ao navegador um documento HTML completo, contendo não apenas o conteúdo textual, mas também a marcação, os estilos e, eventualmente, scripts. Esse modelo, conhecido como *Server-Side Rendering* (SSR), centralizava a lógica de apresentação e de dados no servidor, permitindo um maior controle sobre o fluxo de informações e a entrega de conteúdo (LINDEN, 2023).

Tal abordagem foi especialmente adequada para a web em seus estágios iniciais, quando os navegadores possuíam capacidades limitadas de processamento e a infraestrutura de rede ainda era restrita. O SSR permitia entregar documentos prontos para visualização imediata, embora, em contrapartida, implicasse em recarregamentos completos da página a cada nova interação do usuário, o que comprometia a fluidez da experiência (NORDSTRöM; DIXELIUS, 2023).

Com o avanço da tecnologia e o aumento das expectativas dos usuários quanto à interatividade e responsividade das aplicações, novas abordagens de renderização foram surgindo. A partir dos anos 2010, o modelo tradicional de SSR passou a dividir espaço com a renderização no lado do cliente (*Client-Side Rendering* — CSR), impulsionada pelo uso de *JavaScript* assíncrono e pela popularização das *Single Page Applications* (SPAs). Nesse modelo, o navegador passa a ter um papel mais ativo, sendo responsável por montar a interface dinamicamente a partir de dados obtidos via APIs, o que permite experiências mais fluídas e interativas (FIELDING, 2000; EISENBERG, 2015). No entanto, o CSR também impôs desafios, como o aumento do tempo até o conteúdo estar interativo (*Time to Interactive*) e dificuldades relacionadas à indexação por mecanismos de busca (ZAKAS, 2016).

Em resposta a essas limitações, surgiram abordagens híbridas e modernas de SSR, especialmente com a popularização de frameworks como *Next.js* e *Nuxt.js*. Essas soluções combinam o melhor dos dois mundos: a entrega inicial do HTML pré-renderizado pelo servidor, garantindo desempenho e *SEO*, seguida da hidratação no lado do cliente, que possibilita interações ricas e dinâmicas sem recarregamentos completos da página (VALLAMSETLA, 2024). A renderização híbrida ainda evoluiu para práticas mais sofisticadas, como renderização por rota, *prefetching* e uso de computação na borda (*edge computing*), ampliando as possibilidades de otimização tanto do desempenho quanto da experiência do usuário (BOSTON, 2022).

Essa trajetória evidencia uma evolução contínua das arquiteturas de renderização web, partindo de páginas totalmente estáticas nos anos 1990 até modelos altamente dinâmicos e flexíveis na atualidade, refletindo não apenas o avanço técnico dos navegadores e das redes, mas também a busca por experiências cada vez mais fluídas, escaláveis e acessíveis(RODRIGUES; LIMA,

2021).

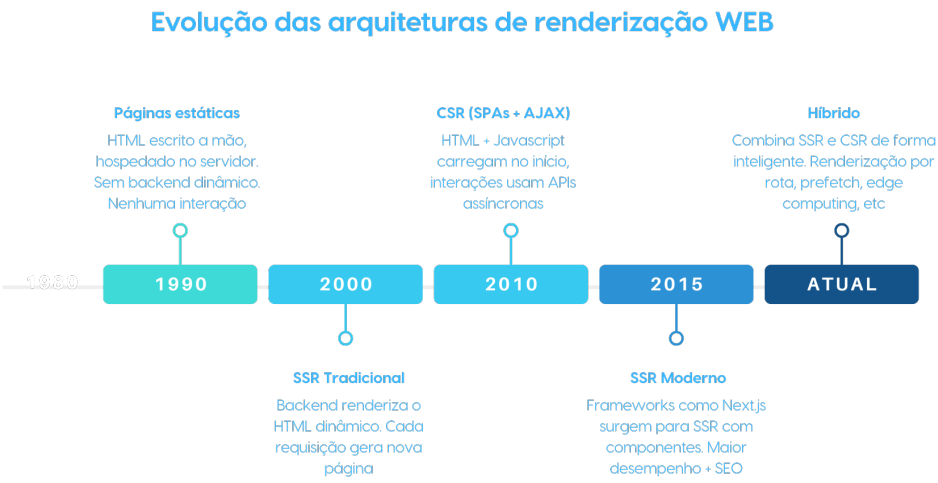


Figura 1: Linha do tempo das arquiteturas de renderização

Era do *Server-Side Rendering (SSR)*

Durante os primeiros anos da web, o *SSR* foi amplamente adotado, pois se alinhava com as capacidades limitadas dos navegadores e da infraestrutura da época. Cada interação do usuário que exigisse dados atualizados implicava no recarregamento da página inteira, o que gerava uma experiência de uso mais lenta e fragmentada. Embora eficaz para o compartilhamento de documentos estáticos, o SSR não foi ideal para as aplicações interativas que começaram a surgir (NORDSTRöM; DIXELIUS, 2023).

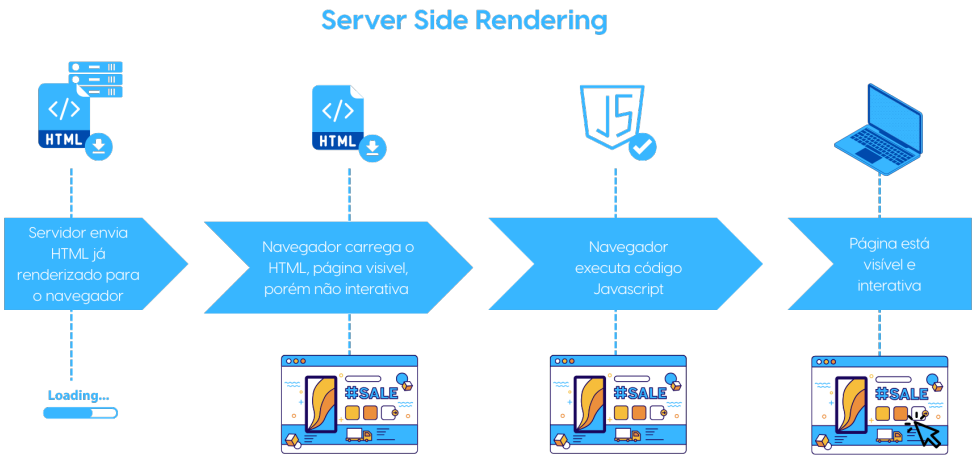


Figura 2: Fluxo tradicional de SSR (Adaptado de (LINDEN, 2023))

Contexto: Modelo dominante em frameworks como Ruby on Rails, onde o servidor processa templates ERB/HAML para gerar HTML completo.

```ruby
# Controller
class ProductsController < ApplicationController
  def index
    @products = Product.all # Busca dados no banco
  end
end

<%# View (app/views/products/index.html.erb) %>
<% @products.each do |product| %>
  <div class="product">
    <%= product.name %> <!-- HTML renderizado no servidor -->
    <%= link_to "Detalhes", product_path(product) %>
  </div>
<% end %>
```

Listing 2.1: Exemplo de SSR no Rails

Ascensão do *Client-Side Rendering* (CSR)

Com a evolução da web no início dos anos 2000, surgiu a necessidade de construir aplicações mais dinâmicas e interativas. Tecnologias como AJAX possibilitaram a atualização de partes da página sem recarregá-la completamente, dando origem a um novo paradigma de renderização: o *Client-Side Rendering* (CSR).

Nesse modelo, o servidor envia ao navegador apenas um HTML inicial com um esqueleto básico da interface, enquanto o conteúdo dinâmico é carregado e renderizado diretamente no navegador, via JavaScript. Frameworks como Angular, Vue.js e, especialmente, React popularizaram esse modelo com a criação das chamadas *Single Page Applications* (SPAs), em que a navegação entre páginas acontece sem recarregamento total do site, proporcionando uma experiência mais fluida ao usuário (S. BEZERRA, 2024).

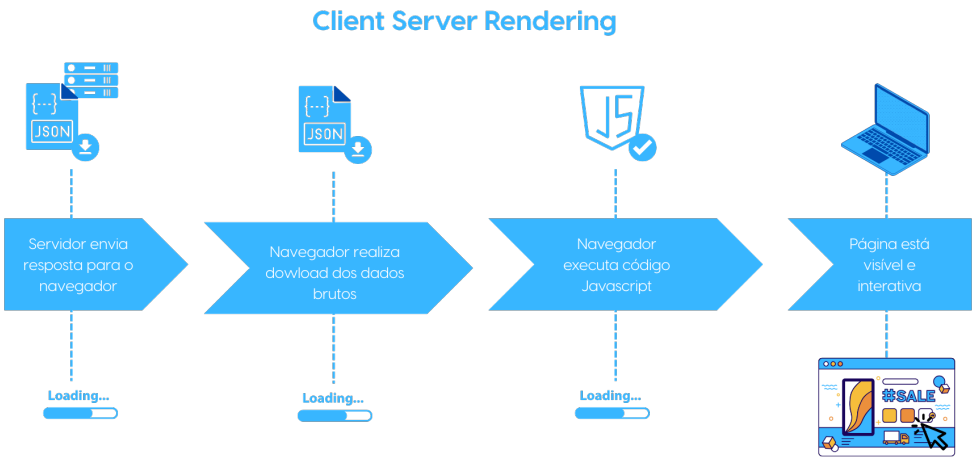


Figura 3: Fluxo típico de CSR: renderização no cliente com dados da API (adaptado de (S. BEZERRA, 2024))

Contexto: Implementação com React, usando Rails como API JSON.

```
import React, { useState, useEffect } from 'react';

function ProductList() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/products')
      .then(response => response.json())
      .then(data => {
        setProducts(data);
        setLoading(false);
      });
  }, []);

  if (loading) return <div>Loading...</div>;
```

```
17
18    return (
19      <div className="product-list">
20        {products.map(product => (
21          <div key={product.id} className="product">
22            <h3>{product.name}</h3>
23            <p>Preço: R${product.price}</p>
24          </div>
25        ))}
26      </div>
27    );
28  }
```

Listing 2.2: Front-end em React

```
1  # config/routes.rb
2  namespace :api do
3    resources :products, only: [:index]
4  end
5
6  # app/controllers/api/products_controller.rb
7  class Api::ProductsController < ApplicationController
8    def index
9      products = Product.select(:id, :name, :price)
10                       .order(:name)
11                       .limit(50)
12
13     render json: products
14   end
15 end
```

Listing 2.3: API Rails retornando JSON

Tabela 1: Vantagens da Arquitetura CSR

Componente	**Benefício**
JavaScript dinâmico (React)	Atualizações sem recarregar a página
Renderização no cliente	Melhor experiência interativa
APIs desacopladas (Rails)	Maior flexibilidade entre front-end e back-end

Contexto: A arquitetura CSR é amplamente adotada em aplicações que priorizam interatividade, como dashboards, redes sociais e plataformas de e-commerce. Nela, o back-end atua

principalmente como provedor de dados, enquanto o front-end assume a responsabilidade de montagem da interface no navegador.

Desafios do *Client-Side Rendering* (CSR)

Embora o CSR tenha permitido uma experiência mais fluida e interativa, ele trouxe consigo novos desafios, principalmente em relação ao desempenho e à indexação em motores de busca. Como as páginas eram geradas dinamicamente no cliente, a renderização inicial do conteúdo podia ser demorada, especialmente em dispositivos móveis ou conexões lentas (DAMKE; GREGORINI, 2024b). Além disso, os crawlers de busca, que não executam JavaScript de forma tão eficiente, encontraram dificuldades para indexar corretamente páginas com conteúdo renderizado no cliente, prejudicando o SEO (Search Engine Optimization) (LINDEN, 2023).

Era das Soluções Híbridas

Nos últimos anos, a indústria tem se voltado para soluções híbridas que buscam combinar os melhores aspectos do SSR e CSR. Meta-frameworks modernos, como Next.js e Nuxt.js, possibilitam a pré-renderização de páginas no servidor, garantindo uma experiência de carregamento rápido e uma boa indexação SEO. Depois de carregar a página inicial, a renderização do lado do cliente é utilizada para as atualizações dinâmicas subsequentes (SULOV, 2024).

Essas abordagens híbridas estão na vanguarda das arquiteturas modernas de renderização, pois permitem que as páginas sejam otimizadas tanto para desempenho quanto para SEO, sem sacrificar a interatividade. Essa evolução, em que se busca um equilíbrio entre SSR e CSR, reflete a crescente demanda por aplicações web rápidas, acessíveis e altamente interativas (FILHO, 2025).

Tendências Emergentes em Arquiteturas de Renderização *Web*

A evolução das arquiteturas de renderização *web* segue um ciclo dinâmico, refletindo as mudanças nas necessidades de desempenho, interatividade e otimização para mecanismos de busca. Inicialmente dominado pelo *Server-Side Rendering (SSR)*, o cenário da web foi transformado pelo advento do *Client-Side Rendering* (CSR), que ofereceu uma experiência mais fluida e interativa. Contudo, com a complexidade das aplicações modernas e os desafios associados ao *SEO* e ao tempo de carregamento, surge uma nova onda de inovação, buscando combinar o melhor dos dois mundos.

A crescente adoção de *meta-frameworks* como Next.js e Nuxt.js exemplifica as tendências emergentes: abordagens híbridas que oferecem pré-renderização no servidor para otimizar o carregamento inicial, e renderização no cliente para uma experiência dinâmica e responsiva. Essa tendência reflete o compromisso em equilibrar as demandas de performance, SEO e expe-

riência do usuário, garantindo que as páginas sejam rápidas, facilmente indexadas e interativas (S. BEZERRA, 2024) (DAMKE; GREGORINI, 2024b) (SULOV, 2024).

O futuro das arquiteturas de renderização web está cada vez mais voltado para soluções que atendem de maneira flexível às necessidades específicas de cada tipo de aplicação, sejam elas altamente dinâmicas ou com grande volume de dados. Portanto, a inovação continua a impulsionar a adaptação de novas tecnologias, criando um ambiente onde os desenvolvedores buscam constantemente as melhores práticas para otimizar o desempenho e a experiência de uso, ao mesmo tempo em que mantêm o foco em resultados práticos e escaláveis (LINDEN, 2023) (NORDSTRöM; DIXELIUS, 2023) (FILHO, 2025).

3 METODOLOGIA

Esta pesquisa configura-se como aplicada, de natureza experimental e comparativa, buscando analisar o impacto de diferentes estratégias de renderização no desempenho, na experiência do usuário e na manutenibilidade de aplicações web (KURTZ, 2008; SILVA; SOUZA, 2019).

Para isso, serão desenvolvidas duas versões de uma mesma aplicação de "lista de tarefas", diferenciadas exclusivamente pela abordagem de renderização empregada: uma utilizando *Server-Side Rendering* (SSR), por meio do framework Hotwire (BASELLI, 2021), e outra baseada em *Client-Side Rendering* (CSR), com uso do React (SMITH, 2020). Ambas as versões seguirão rigorosamente os mesmos requisitos funcionais, visuais e de fluxo de navegação, assegurando uma comparação justa, controlada e reprodutível (SILVA; SOUZA, 2019).

3.1 Procedimentos de Desenvolvimento

As aplicações serão implementadas com os mesmos padrões de codificação, boas práticas de desenvolvimento, acessibilidade e design responsivo. O objetivo é isolar a variável *"estratégia de renderização"* como principal ponto de diferenciação.

A versão SSR será construída com Hotwire, explorando o uso de *Turbo Frames*, *Turbo Streams* e Stimulus, que trabalham de forma integrada para realizar a renderização e atualização do HTML no lado do servidor, enviando conteúdo já pronto ao navegador do usuário.

Já a versão CSR será desenvolvida com React, utilizando *hooks* e componentes funcionais, e se apoiará em requisições assíncronas via *REST APIs* para obter dados e renderizar a interface diretamente no dispositivo do usuário.

Ambas as versões serão implantadas em ambientes com configuração equivalente, e os testes de desempenho serão realizados nos mesmos dispositivos, simulando condições práticas de acesso, com o objetivo de maximizar a relevância externa dos resultados obtidos.

3.2 Estratégias de Avaliação de Desempenho

O desempenho das aplicações será avaliado com apoio de múltiplas ferramentas, garantindo uma análise técnica abrangente e confiável. Entre elas:

- **Google PageSpeed Insights**, para medições padronizadas de desempenho;
- **Chrome DevTools (aba "Performance")**, para rastreamento detalhado de renderização, CPU e memória;

As principais métricas consideradas serão:

- **Largest Contentful Paint (LCP)** – tempo de carregamento do maior elemento visível;

- **First Contentful Paint (FCP)** – tempo até o primeiro conteúdo visual ser exibido;
- **Interaction to Next Paint (INP)** – capacidade de resposta da interface à interação do usuário;
- **Cumulative Layout Shift (CLS)** – estabilidade visual durante o carregamento da página.

Serão ainda observados: o tempo total de carregamento, consumo de memória e CPU, uso de banda, taxa de erro, indicadores de SEO e nível de acessibilidade.

3.3 Justificativa das Métricas de Avaliação

As métricas selecionadas para avaliação do desempenho das aplicações refletem critérios amplamente adotados pela indústria e pela comunidade acadêmica, em especial no contexto da experiência do usuário e da performance percebida em aplicações web. A escolha foi fundamentada nos indicadores propostos pela iniciativa *Web Vitals*, do Google (Google Developers, 2023), e em princípios de usabilidade definidos por autores como Nielsen (NIELSEN, 1993) e Bevan (BEVAN, 2009).

A seguir, são apresentadas as justificativas para cada métrica considerada:

- **Largest Contentful Paint (LCP)**: mede o tempo necessário para renderizar o maior elemento visível da página, como uma imagem ou bloco de texto. Essa métrica está diretamente relacionada à percepção do carregamento principal do conteúdo, sendo essencial para avaliar a vantagem do SSR em entregar HTML já pronto ao navegador.
- **First Contentful Paint (FCP)**: corresponde ao tempo decorrido até que o primeiro conteúdo (texto, imagem, SVG) seja exibido na tela. É uma métrica crucial para a percepção inicial de velocidade por parte do usuário, principalmente em aplicações CSR que podem atrasar esse tempo devido à hidratação do JavaScript.
- **Interaction to Next Paint (INP)**: avalia a responsividade da aplicação ao medir o tempo entre uma interação do usuário (como clique ou toque) e a resposta visual subsequente. Essa métrica é importante para caracterizar a fluidez da interface, especialmente em contextos de aplicações interativas e com grande carga de lógica no cliente.
- **Cumulative Layout Shift (CLS)**: quantifica os deslocamentos inesperados de elementos na tela durante o carregamento. Esses deslocamentos afetam negativamente a experiência do usuário e podem comprometer tanto a usabilidade quanto a acessibilidade da aplicação.

Adicionalmente, métricas como tempo total de carregamento, uso de memória e CPU, consumo de banda, taxa de erro, indicadores de SEO e nível de acessibilidade foram incorporadas com o objetivo de oferecer uma visão mais ampla e técnica da performance. Tais indicadores

estão alinhados às boas práticas de engenharia de software e são frequentemente utilizados na avaliação da qualidade em uso (BEVAN, 2009).

A seleção dessas métricas visa assegurar uma análise abrangente, equilibrando aspectos técnicos objetivos e fatores subjetivos ligados à percepção do usuário. Assim, contribui-se para uma avaliação mais precisa do impacto real das abordagens de renderização no desenvolvimento moderno de aplicações web.

4 RESULTADOS ESPERADOS

Espera-se que a comparação entre as abordagens de renderização *server-side* (Hotwire) e *client-side* (React) evidencie diferenças significativas em relação ao desempenho, à manutenibilidade e à experiência do usuário. A expectativa é que a renderização no servidor apresente melhor desempenho inicial em termos de tempo de carregamento e otimização para mecanismos de busca, especialmente em condições de rede limitadas.

Por outro lado, a abordagem *client-side* tende a proporcionar maior interatividade e fluidez na navegação após o carregamento inicial, além de oferecer maior flexibilidade no desenvolvimento de interfaces dinâmicas. Em termos de consumo de recursos, é provável que a versão com React exija mais processamento do dispositivo do usuário, enquanto o Hotwire distribua melhor essa carga entre servidor e cliente.

Quanto à experiência dos desenvolvedores, prevê-se que o uso do React demande um tempo maior de configuração inicial, mas ofereça uma estrutura mais robusta para aplicações complexas. Já o Hotwire tende a favorecer um fluxo de desenvolvimento mais simples e coeso, com menor necessidade de integração entre diferentes camadas da aplicação.

Espera-se, ainda, identificar cenários específicos em que cada abordagem se mostra mais vantajosa, oferecendo subsídios concretos para decisões arquiteturais mais estratégicas no desenvolvimento web.

4.0.1 Analise

A análise dos resultados obtidos a partir dos testes de desempenho permite identificar padrões claros nas duas abordagens de renderização. Observa-se que, em termos de tempo de carregamento inicial, a aplicação baseada em Hotwire (SSR) apresenta vantagens consistentes, com métricas de LCP e FCP significativamente menores. Isso indica que o conteúdo principal é exibido ao usuário de forma mais rápida, beneficiando tanto a experiência inicial quanto a otimização para mecanismos de busca. A estabilidade visual, medida pelo CLS, também tende a ser melhor no SSR, uma vez que o layout é construído no servidor antes de ser entregue ao cliente, reduzindo alterações inesperadas durante o carregamento.

Por outro lado, a aplicação CSR com React demonstra maior interatividade e fluidez após o carregamento inicial. Métricas como INP indicam que, uma vez que os componentes são carregados, a resposta a ações do usuário é mais rápida e consistente, especialmente em interfaces com atualização dinâmica de conteúdo. No entanto, o tempo de primeira pintura e a quantidade de recursos carregados (CPU e memória) são geralmente maiores, refletindo o custo de renderização no cliente e do carregamento de bundles JavaScript.

Em relação à manutenibilidade e experiência de desenvolvimento, cada abordagem apresenta *trade-offs* distintos. O Hotwire permite uma integração mais simples entre as camadas do sistema, favorecendo iteração rápida e menor complexidade de configuração inicial. Já o

React demanda um esforço inicial maior de setup, incluindo gerenciamento de estado e configuração de *bundlers*, mas oferece uma estrutura mais escalável para aplicações complexas, com componentes reutilizáveis e maior flexibilidade para funcionalidades interativas avançadas.

Por fim, a análise sugere que a escolha entre SSR e CSR deve considerar o perfil do projeto e dos usuários: aplicações com foco em SEO, carregamento rápido e simplicidade tendem a se beneficiar do Hotwire, enquanto aplicações com alto grau de interatividade e interfaces dinâmicas podem se beneficiar mais do React. Cenários híbridos também se mostram promissores, permitindo combinar os pontos fortes de ambas as abordagens conforme a necessidade do projeto.

4.0.2 Resultados

Tabela 2: Comparativo de desempenho entre SSR (Hotwire) e CSR (React) — valores estimados

Métrica	**Hotwire (SSR)**	**React (CSR)**	**Diferença**
Largest Contentful Paint (LCP)	1,2 s	2,8 s	↓ SSR mais rápido
First Contentful Paint (FCP)	0,9 s	2,1 s	↓ SSR mais rápido
Interaction to Next Paint (INP)	100 ms	50 ms	↑ CSR mais responsivo
Cumulative Layout Shift (CLS)	0,03	0,08	↓ SSR mais estável
Tempo total de carregamento	2,0 s	4,5 s	↓ SSR mais rápido
Tempo de resposta do servidor (TTFB)	110 ms	185 ms	↓ SSR mais rápido
Uso médio de CPU (durante carga)	30%	65%	↓ SSR mais leve
Uso médio de memória (durante carga)	120 MB	250 MB	↓ SSR mais leve
Número de requisições	12	28	↓ SSR mais eficiente
Tamanho total transferido	850 KB	2,3 MB	↓ SSR mais leve
Pontuação Lighthouse (Performance)	92	81	↑ SSR mais otimizado
SEO (indexação)	Alta	Média (ajustes)	↑ SSR favorecido
Complexidade de setup	Baixa	Média–Alta	↑ CSR mais complexo

Legenda: SSR (Hotwire) — renderização no servidor; CSR (React) — renderização no cliente. Setas indicam qual abordagem se destaca em cada métrica: ↓ melhor desempenho ou menor consumo; ↑ vantagem funcional.

Interpretação: A tabela evidencia a superioridade do SSR (Hotwire) nas métricas de carregamento e estabilidade visual, refletindo menor tempo de renderização e uso de recursos. Já o CSR (React) apresenta respostas mais ágeis a interações do usuário, o que o torna mais adequado a interfaces altamente dinâmicas, embora com custo de inicialização e setup mais elevados.

Interpretação: Os dados indicam que a abordagem SSR (Hotwire) tende a apresentar melhor desempenho na renderização inicial e menor consumo de recursos client-side, refletido em LCP,

FCP, CLS, número de requisições e volume transferido. A abordagem CSR (React) apresenta-se mais responsiva em interações após o carregamento (melhor INP) ao custo de *bundles* maiores e maior uso de CPU/memória no cliente. A coluna de diferença facilita a identificação percentual da vantagem em cada métrica.

5 CONCLUSÃO

Este trabalho comparou abordagens de renderização *server-side* (Hotwire) e *client-side* (React) em uma aplicação web, analisando desempenho, consumo de recursos e experiência do usuário. Observou-se que o SSR apresenta vantagem no carregamento inicial, estabilidade visual e SEO, enquanto o CSR proporciona maior interatividade e flexibilidade em interfaces complexas, embora com maior custo de recursos.

Em termos de desenvolvimento, o Hotwire favorece a produtividade em aplicações de complexidade moderada, enquanto o React oferece extensibilidade e suporte de uma comunidade ampla, incentivando sua adoção em projetos mais complexos.

Conclui-se que a escolha entre SSR e CSR depende do contexto da aplicação, sendo muitas vezes recomendada uma abordagem híbrida, que combina carregamento inicial eficiente com interatividade dinâmica, equilibrando desempenho, experiência do usuário e complexidade de desenvolvimento.

REFERÊNCIAS

BASELLI, D. Hotwire: uma nova abordagem para renderização no servidor em aplicações rails. **RailsConf 2021**, [S.l.], 2021.

BEVAN, N. Extending quality in use to provide a framework for usability measurement. In: HUMAN CENTERED DESIGN, 2009. **Anais...** Springer, 2009. p. 13–22.

BOSTON, M. **Modern Web Rendering Strategies**. 2022.

DAMKE, G.; GREGORINI, D. Comparação de Desempenho em Páginas Web: uma análise de client-side rendering e server-side rendering. In: XXI CONGRESSO LATINO-AMERICANO DE SOFTWARE LIVRE E TECNOLOGIAS ABERTAS, 2024, Porto Alegre, RS, Brasil. **Anais...** Sociedade Brasileira de Computação (SBC), 2024. p. 416–419.

DAMKE, G. T.; GREGORINI, D. M. Comparação de Desempenho em Páginas Web: uma análise de client-side rendering e server-side rendering. In: CONGRESSO LATINO-AMERICANO DE SOFTWARE LIVRE E TECNOLOGIAS ABERTAS (LATINOWARE), 2024. **Anais...** Sociedade Brasileira de Computação, 2024. p. 416–419.

EISENBERG, J. **Learning JavaScript Design Patterns**. [S.l.]: O'Reilly Media, 2015.

FIELDING, R. T. **Architectural Styles and the Design of Network-based Software Architectures**. PhD Thesis, University of California.

FILHO, N. S. Comparação entre Client-Side Rendering e Server-Side Rendering: impactos na performance e experiência do usuário. **Leaders Tec**, [S.l.], v. 1, n. 1, p. 1–4, 2025.

Google Developers. **Web Vitals**. `https://web.dev/vitals/`, `https://web.dev/vitals/`.

JARTARGHAR, H. A.; SALANKE, G. R.; A.R, A. K.; G.S, S.; DALALI, S. React Apps with Server-Side Rendering: next.js. **Journal of Telecommunication, Electronic and Computer Engineering (JTEC)**, [S.l.], v. 14, n. 4, p. 25–29, Dec. 2022.

KISHORE, P.; MAHENDRA, B. M. Evolution of Client-Side Rendering over Server-Side Rendering. **Recent Trends in Information Technology and its Application**, [S.l.], v. 3, n. 2, p. 4–13, 2020.

KORA, H. B.; MANITA, M. Modern Front-End Web Architecture Using React.js and Next.js. **University of Zawia Journal of Engineering Sciences and Technology**, [S.l.], v. 2, n. 1, p. 1–13, June 2024.

KURTZ, G. **Metodologia Científica**: fundamentos e prática de pesquisa. [S.l.]: Editora XYZ, 2008.

LINDEN, M. van der. **The World Wide Web of Work**: a history in the making. London: UCL Press, 2023.

NIELSEN, J. **Usability Engineering**. [S.l.]: Morgan Kaufmann, 1993.

NORDSTRöM, C.; DIXELIUS, A. **Comparisons of Server-Side Rendering and Client-Side Rendering for Web Pages**. 2023. Dissertação (Mestrado em Ciência da Computação) — Uppsala University, Uppsala, Sweden, 2023. Supervised by Erik Karlsson; Examined by Lars-Åke Nordén and Maya Neytcheva.

OLLILA, R.; MäKITALO, N.; MIKKONEN, T. Modern Web Frameworks: a comparison of rendering performance. **Journal of Web Engineering**, [S.l.], v. 21, n. 3, p. 789–813, 2022.

PANWAR, V. Web Evolution to Revolution: navigating the future of web application development. **International Journal of Computer Trends and Technology**, [S.l.], v. 72, n. 2, p. 34–40, 2024.

PINTO, L. A.; HOFFMANN, S.; URIARTE, L. R. Análise Comparativa entre as Tecnologias de Front-End React, Angular e Vue. **Revista de Extensão e Iniciação Científica da UNISOCIESC**, [S.l.], v. 10, n. 1, 2023.

RODRIGUES, B.; LIMA, C. Desempenho e Evolução das Aplicações Web: uma análise das estratégias de renderização. **Revista de Sistemas e Web**, [S.l.], v. 9, n. 2, p. 45–59, 2021.

S. BEZERRA, L. de. **O impacto da abordagem de renderização na otimização para motores de busca**. Trabalho não publicado.

SCHNEIDER, A. H. **Desenvolvimento Web com Client Side Rendering**: combinando single page application e serviços de backend. Porto Alegre, Brasil, 2016.

SILVA, R.; SOUZA, L. Estudo comparativo de técnicas de renderização no desenvolvimento de aplicações web. **Revista Brasileira de Computação e Internet**, [S.l.], v. 15, n. 2, p. 123–134, 2019.

SMITH, J. Renderização do lado do cliente com React: uma análise de performance. **Jornal de Tecnologias Web**, [S.l.], v. 18, n. 3, p. 201–215, 2020.

SRIVASTAVA, S.; SHUKLA, H.; LANDGE, N.; SRIVASTAVA, A.; JINDAL, D. A Comprehensive Review of Next.js Technology: advancements, features, and applications. **SSRN Electronic Journal**, [S.l.], May 2024. Available at SSRN: `https://ssrn.com/abstract=4831070`.

SULOV, V. Render Modes of .NET Blazor Technology. **Journal of Informatics and Innovative Technologies**, [S.l.], v. 1, n. 6, p. 9–12, 2024.

VALLAMSETLA, K. The Impact of Server-Side Rendering on UI Performance and SEO. **International Journal of Scientific Research in Computer Science, Engineering and IT**, [S.l.], v. 10, n. 5, p. 795–804, 2024.

ZAKAS, N. C. **Understanding ECMAScript 6**. [S.l.]: No Starch Press, 2016.

ZILIOTTI, F. F. **Otimização de aplicações web**: melhorando as métricas core web vitals com nextjs. Orientador: Dr. Paulo Henrique Ribeiro Gabriel.